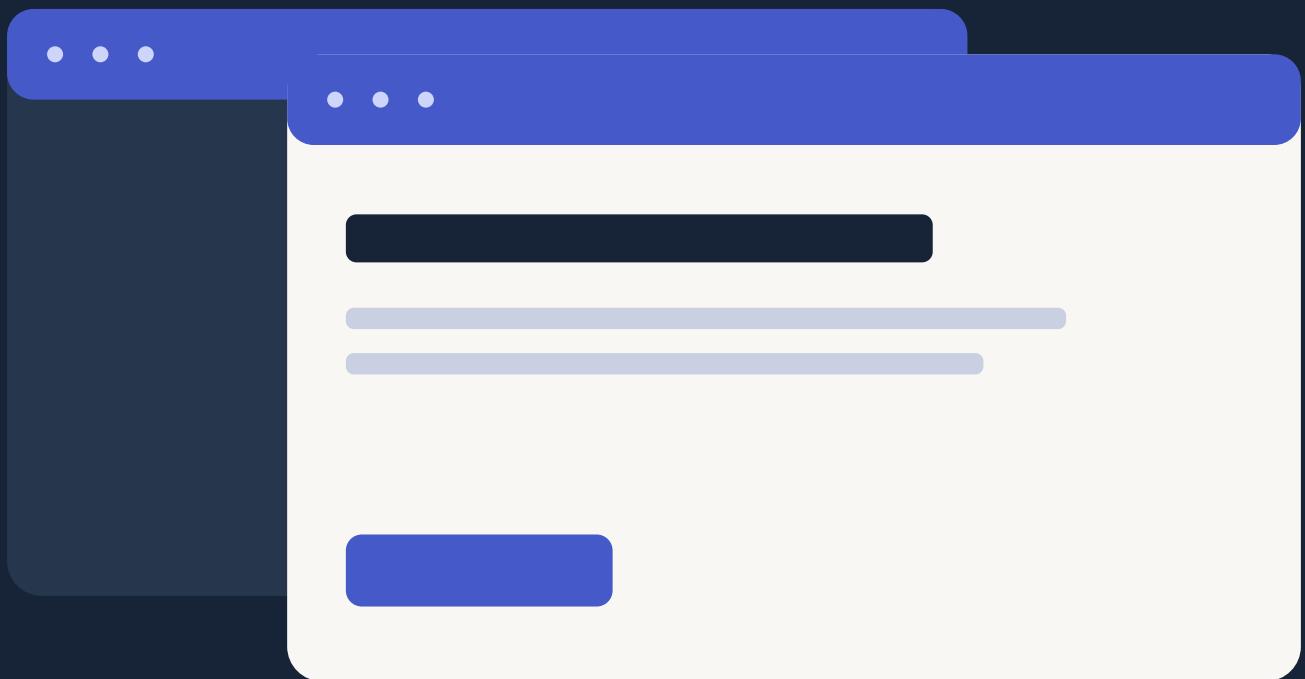


Web Development Complete Guide

Build your first real website, use AI responsibly, test it,
and publish it.



START HERE

A useful outcome in 30 days

One project. Clear milestones. A site you can explain and improve.

This book teaches you to build a responsive website for an illustrative Surat-based service business, “Nila Studio.” The business and its content are fictional; replace them with your own details before publishing. You will write the core code, use AI as a reviewer and helper, and learn to verify every result.

Your finished deliverable

A three-section service landing page with clear navigation, services, a contact path, mobile layout, basic accessibility and SEO, a tested deployment, and a simple handoff note.

What this guide covers

HTML structure, CSS layout, JavaScript interaction, Git, publishing, testing, performance, safe forms, and practical AI prompts. It also shows when a static site is enough and when you need a back end.

What to bring

A laptop, a modern browser, a text editor such as VS Code, an email address for a code-hosting account, and time to build in small sessions. The examples use plain files so you can understand them without a framework.

Reality check

Thirty days is a practice plan, not a promise of job readiness or earnings. A public website needs careful review of its text, permissions, privacy handling, hosting settings, and client approval.

NAVIGATION

Inside this guide

01 Map the web

Pages 04–06 • Roles, scope, and tools

02 Build the foundation

Pages 07–11 • HTML, CSS, responsive layout, JavaScript

03 Use AI with judgment

Pages 12–14 • Prompts, verification, debugging

04 Make it production-minded

Pages 15–19 • Accessibility, speed, SEO, forms, security

05 Ship and improve

Pages 20–24 • Git, deployment, QA, back end, handoff

06 Practice system

Pages 25–27 • Action plan and quick reference

How to read

Build the project as you read. “Try it” instructions are exercises; “Illustrative Example” labels identify invented business details. Platform settings can change, so verify the linked official guides before deployment.

01 / MAP THE WEB

What happens when a page loads?

The mental model behind every tool choice.

1 Address

A URL names a resource.

2 Request

The browser asks a server.

3 Response

Files arrive over HTTPS.

4 Render

HTML, CSS and JS run.

Three layers you control

HTML gives content meaning. CSS controls presentation and layout. JavaScript responds to events and changes the page. A server or API becomes necessary when you must safely store or process private data.

The smallest useful build

Start with an index.html file, a style.css file, and a script.js file. Open the page locally, inspect it in browser developer tools, and save each meaningful change in Git.

Try it

Explain the flow above aloud: “When someone opens my site, the browser requests my files, builds a document, applies styles, and runs scripts.” If you cannot explain a feature in those terms, simplify it first.

01 / SCOPE

Choose the right kind of website

Need	Good starting point	Why
Portfolio, brochure, event page	Static HTML/CSS/JS	Content and simple interactions can be published as files.
Regular articles or nontechnical edits	CMS or site builder	Editors can update content without changing code.
Accounts, private data, payments	Full stack / managed service	Requires trusted server-side logic, data handling and security.

Project brief: Nila Studio (Illustrative Example)

A small design service wants inquiries from local businesses in Surat. Audience: founders who need a simple brand identity. Primary action: contact the studio. Three site sections: Home, Services, Contact. Secondary proof: a small process description, not invented testimonials.

One-page scope worksheet

Audience: _____ • Main problem: _____ • Main action: _____
Three sections: _____ / _____ / _____
Content owner: _____ • Contact method: _____ • Deadline: _____

A boundary that prevents rework

Write real text and collect approved images before polishing the layout. For a client, agree on number of pages, revisions, content responsibility, hosting ownership, domain ownership, and who receives form submissions.

01 / TOOLS

A lean beginner toolkit

Job	Tool / resource	Use it for
Edit code	VS Code or another code editor	Files, search, formatting, terminal.
Test in browser	Browser developer tools	Responsive view, console, network and elements.
Learn standards	MDN Web Docs	HTML, CSS, JavaScript and examples.
Track changes	Git + GitHub	Save versions, share and publish a static site.
Review quality	Lighthouse / PageSpeed Insights	Find performance, accessibility and SEO issues.
AI assistant	Your chosen coding assistant	Draft plans, explain errors, review small changes.

All recommendations above describe categories or examples, not a pricing promise. Check the current service terms and account requirements before committing a client project.

Folder map

Create one folder named nila-site. Keep the entry page and related files together. Assets such as approved photos go in an assets folder with descriptive filenames.

```
nila-site/  
  index.html  
  style.css  
  script.js  
  assets/
```

02 / HTML

Write the content before the decoration

A page should make sense even with styling switched off.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Nila Studio | Brand Design in Surat</title>
  <meta name="description" content="Brand design for growing
  businesses in Surat. Explore our services and contact us.">
  <link rel="stylesheet" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <header><nav aria-label="Main navigation">...</nav></header>
  <main>...</main>
  <footer>...</footer>
</body>
</html>
```

The rule of meaning

Use header, nav, main, section, and footer for their roles. Use headings in a logical order. A button performs an action; an anchor takes a person to a destination. This makes your page clearer for people, assistive technology, and future editors.

Try it

Create index.html and paste the skeleton. Replace “...” with plain text first. Open the file in a browser. Confirm the title is visible in the tab and your content appears in reading order.

02 / HTML

A useful landing page structure

```
<nav aria-label="Main navigation">
  <a href="#home">Nila Studio</a>
  <a href="#services">Services</a>
  <a href="#contact">Contact</a>
</nav>
<main>
  <section id="home">
    <h1>Brand design for growing businesses</h1>
    <p>Clear visual identity for your next stage.</p>
    <a href="#contact">Discuss a project</a>
  </section>
  <section id="services">
    <h2>What we can help with</h2>
    <ul><li>Logo systems</li><li>Social templates</li></ul>
  </section>
  <section id="contact">
    <h2>Start a conversation</h2>
    <a href="mailto:hello@example.com">Email us</a>
  </section>
</main>
```

The business name, service offer and email are Illustrative Examples. Replace them with approved details. An email link is a working contact path without collecting form data on your site.

Content check

Can a first-time visitor answer: What is offered? Who is it for? Where is it available? What should I do next? If the answer is buried, rewrite the hero before adding more sections.

02 / CSS

Style for clarity, then personality

```
:root {
  --ink: #172336; --accent: #4659c9;
  --surface: #f8f7f3;
}
* { box-sizing: border-box; }
body { margin: 0; color: var(--ink);
  background: var(--surface);
  font: 1rem/1.6 system-ui, sans-serif; }
nav, main, footer { width: min(100% - 2rem, 70rem);
  margin-inline: auto; }
nav { display: flex; gap: 1.5rem; align-items: center;
  padding-block: 1.25rem; flex-wrap: wrap; }
section { padding-block: clamp(3rem, 7vw, 6rem); }
h1 { max-width: 15ch; font-size: clamp(2.2rem, 6vw, 4.5rem);
  line-height: 1.08; }
a { color: var(--accent); }
a:focus-visible { outline: 3px solid var(--accent);
  outline-offset: 4px; }
```

Read CSS in four passes

Select an element; choose the property; set a value; see what changed. Use browser developer tools to inspect the box model when spacing feels wrong. Change one rule at a time.

Try it

Paste the CSS into style.css. Resize the browser to a narrow width. The heading should scale, and the navigation should wrap without hiding links.

02 / RESPONSIVE

Design for narrow screens first

The mobile-first sequence

At a small width, make content readable in one column. Then add layout only when there is room. Test at several widths by dragging the browser, not only at one preset device.

```
.services-grid { display: grid; gap: 1rem; }  
.service-card { padding: 1.5rem; background: white;  
  border: 1px solid #d7dde6; border-radius: .75rem; }  
@media (min-width: 48rem) {  
  .services-grid { grid-template-columns: repeat(2, 1fr); }  
}
```

Four checks on an actual phone

1. Can you read without zooming? 2. Are tap targets easy to use? 3. Does text stay within the screen? 4. Can you reach the contact action without a confusing menu?

Image rule

Use approved images and give meaningful images useful alternative text. Decorative images can use empty alt text. Compress large files and avoid putting important text inside an image.

Try it

Turn the services list into two cards inside a services-grid container. Compare at narrow and wide widths. Keep the HTML reading order sensible even when the CSS grid changes.

02 / JAVASCRIPT

Add one real interaction

JavaScript should make a task easier. For this project, add a simple “show more” explanation beneath your process summary. The HTML remains useful when JavaScript is unavailable.

```
<button type="button" id="details-toggle"
  aria-expanded="false" aria-controls="more-details">
  Show the process
</button>
<div id="more-details" hidden>
  <p>We clarify the brief, create directions,
  then prepare agreed final files.</p>
</div>
```

```
const toggle = document.querySelector("#details-toggle");
const details = document.querySelector("#more-details");
if (toggle && details) {
  toggle.addEventListener("click", () => {
    const opening = details.hidden;
    details.hidden = !opening;
    toggle.setAttribute("aria-expanded", String(opening));
    toggle.textContent = opening
      ? "Hide the process" : "Show the process";
  });
}
```

Verify it

Click with a mouse, then press Tab to focus the button and Enter or Space to activate it. Check the browser console for errors. Explain each line before accepting AI suggestions to change it.

03 / AI

Use AI as a pair, not an authority

A repeatable loop for useful assistance.

01 Brief Give the goal, audience, files, and constraints.

02 Ask Request one bounded task and the expected format.

03 Inspect Read every changed line and explain it.

04 Test Run the page; check edge cases and browser console.

05 Revise Report the observed bug, not just “it fails.”

Good uses

Outline information architecture, explain unfamiliar code, generate small draft functions, compare two implementation options, identify likely causes of a reproducible error, and create a test checklist.

Human checks

AI can produce plausible but broken code, inaccessible controls, copied content, insecure handling, or outdated API usage. Review dependencies, permissions, licenses, secrets, and every claim about current platform behavior against official documentation.

Privacy rule

Remove passwords, API keys, private client data and personal customer details before sending code or screenshots to any AI tool. Follow the client’s data-sharing rules.

03 / PROMPT PACK

Five prompts worth saving

1 Plan a page

"I am building a one-page site for [audience]. Goal: [action]. Propose three sections and their exact purpose. Avoid invented proof or statistics. Keep the scope suitable for plain HTML/CSS/JS."

2 Explain code

"Explain this snippet line by line for a beginner. Identify any browser compatibility or accessibility concern. Then ask me one question to check understanding: [snippet]."

3 Debug with evidence

"Expected: [behavior]. Actual: [behavior]. Browser console: [error]. Here are the relevant files: [code]. Give the likely cause, the smallest fix, and three ways to verify it."

4 Review accessibility

"Review this HTML for semantic structure, headings, link names, labels, keyboard use and focus. List issues by impact. Do not invent results from a screen-reader test you did not perform."

5 Improve content

"Rewrite this hero for [audience] with a clear offer and action. Preserve the facts exactly. Do not add claims, awards, customer numbers or testimonials: [copy]."

03 / VERIFICATION

The AI output review sheet

Question	How to verify
Does it run?	Open the page, inspect console, click the feature.
Is it the smallest change?	Review the diff; remove unrelated rewrites.
Does it preserve meaning?	Compare text with the approved brief.
Is it accessible?	Keyboard through the flow; inspect labels and focus.
Is it safe?	Look for secrets, unsafe HTML insertion and data flow.
Is the claim current?	Check official docs before repeating platform details.

A bug report that gets a useful answer

File: script.js. Expected: clicking the button reveals details. Actual: nothing changes.

Reproduction: load page, click button. Console: "Cannot read properties of null." Include the related HTML and JavaScript, plus what you already tried.

Decision rule

Accept a suggestion only after you understand why it works and have seen the intended result.
If the answer changes multiple files unexpectedly, ask for a smaller patch.

04 / ACCESSIBILITY

Make the site usable for more people

A short manual audit

Use headings in a sensible order. Give links descriptive text. Ensure all interactive elements work with a keyboard, show visible focus, and have a clear name. Label form inputs. Provide useful alt text when an image carries information.

Test	Pass looks like
Keyboard	Tab reaches every control in a logical order; Enter/Space work.
Zoom / narrow screen	Content remains available without horizontal scrolling.
Images off	Important meaning remains in text or alt text.
Contrast	Text is readable against its background.
Forms	Each input has a visible or programmatic label and clear errors.

Two useful corrections

Placeholder text is not a substitute for a persistent label. A clickable div is not automatically a button. Prefer native HTML controls whenever they fit.

Try it

Unplug the mouse for five minutes. Navigate from the logo to the contact action and back. Record every point where you lose focus or cannot understand the next step.

04 / PERFORMANCE

Make every byte earn its place

The practical order

First measure a published page with browser tools or PageSpeed Insights. Then address the largest avoidable issue: oversized images, unnecessary scripts, heavy fonts, or layout shifts. Measure again after each change.

If you see...	Try this first
Huge hero image	Resize to display size; use a suitable modern format.
Slow third-party scripts	Remove nonessential tags; load only what is needed.
Layout jumping	Reserve image dimensions and avoid late content injection.
Too many font files	Use fewer families and weights, or a system stack.

Core Web Vitals, in plain English

Largest Contentful Paint concerns when the main visible content appears. Interaction to Next Paint concerns response to user input. Cumulative Layout Shift concerns visual stability. The metrics and thresholds may evolve; use current web.dev guidance when assessing a live site.

Try it

Take a baseline screenshot and note the top three audit findings. Fix one issue at a time. Compare the page in a mobile connection simulation and on a real phone when possible.

04 / DISCOVERY

SEO begins with a helpful page

What to do

Write a unique, accurate title and description. Use one clear main heading. Put services and location in visible page text when relevant. Use readable URLs, descriptive anchor text, and internal links if the site has multiple pages. Ensure your page can be accessed and rendered.

What to avoid

Do not repeat a keyword unnaturally, promise ranking, hide text for search engines, or publish AI-written claims without checking them. A page can be technically sound and still require time and demand to be discovered.

Illustrative Example

Weak: "Best Brand Design | Best Design Surat | Top Brand Design."

Clear: "Nila Studio | Brand Design in Surat."

The clear title describes the page without pretending to be a verified ranking.

A simple content outline

Home: offer and action. Services: what is included and for whom. Process: how inquiries move forward. Contact: approved email or other contact route. Add a privacy notice when you collect personal information.

04 / FORMS

Contact forms need a real destination

A styled form with no handling code does not deliver inquiries. Decide first where submissions go, who can access them, how long data is kept, and what consent or notice is needed. For a first static site, an email link or a carefully configured managed form service may be more appropriate.

If you add a form

Label fields, ask only for necessary information, show clear error and success states, prevent spam, and test the delivery path end to end. Browser validation helps users correct mistakes; a server or provider still must validate and handle data securely.

```
<label for="email">Email address</label>  
<input id="email" name="email" type="email"  
  autocomplete="email" required>
```

Never publish a fake success state

A message that says “Sent” must mean the server or provider accepted the submission. Do not make a button appear functional when inquiries are discarded.

India-first note

If you collect names, phone numbers or other personal data, check current Indian legal requirements and your provider’s terms. For tax, privacy and legal decisions, use official guidance or a qualified professional. This guide is general education, not legal advice.

04 / SAFETY

A first security checklist

- Publish over HTTPS and keep dependencies and hosting settings current.
- Never put API secrets, passwords or private keys in browser JavaScript or a public repository.
- Treat all user input as untrusted. Validate on the server or trusted service.
- Avoid inserting untrusted text as HTML. Prefer safe text APIs when possible.
- Give accounts only the access they need and enable available account protections.
- Back up source files and document who owns the domain and hosting.

A useful distinction

The browser is controlled by the visitor. Anything sent to it can be inspected. Sensitive operations such as account access, payments, and private data storage need trusted server-side or managed-service controls.

Before a client handoff

Confirm approved content, service credentials, domain access, billing ownership, form recipient, backup plan, and renewal responsibilities. Record them in a shared handoff document without exposing secrets in the PDF or repository.

05 / VERSION CONTROL

Save progress without fear

A minimal Git rhythm

Initialize the project, inspect changes, commit a meaningful unit, and push to a remote repository. Use descriptive messages such as “Add responsive services grid” instead of “changes.” Keep credentials and generated secrets out of commits.

```
git init
git add index.html style.css script.js
git commit -m "Build initial landing page"
# Add a remote using the instructions on your hosting service
git push
```

Before you push

Open every changed file, check for sample emails, private notes, API keys and licensed assets. If a secret was committed, removing it in a later commit does not automatically make it safe; revoke or rotate it with its provider.

Try it

Make three small commits: page structure, responsive styling, and the disclosure interaction. Open the commit history and describe what changed in each.

05 / PUBLISH

Take the static site live

For this plain-file project, GitHub Pages is one documented publishing option. Follow its current quickstart: create a repository, push the files, choose a publishing source in Pages settings, and open the published URL. Repository visibility, plan eligibility and interface labels can change; check the linked documentation.

Live-site verification

Open the exact published address on a phone and laptop. Confirm CSS and JavaScript load, anchors work, email links open, and no sample content remains. Inspect the browser console. Run a quality audit against the live URL, not only the local file.

Custom domain later

A domain is optional for practice. For client work, document who owns the domain, where DNS is managed, renewal dates, HTTPS status, and access recovery. Do not put the domain in your own personal account without an explicit agreement.

Deployment checklist

- ☐ Correct title and contact details
- ☐ Approved assets
- ☐ All links tested
- ☐ Keyboard test
- ☐ Phone test
- ☐ No secrets
- ☐ HTTPS
- ☐ Live URL saved
- ☐ Owner and update process documented

05 / QA

Test the journey, not just the layout

Scenario	Action	Expected result
New visitor	Read hero and services	Offer and audience are clear.
Keyboard user	Tab to contact action	Focus is visible and order makes sense.
Narrow screen	Open at phone width	No clipped content or sideways scroll.
Slow connection	Load the live page	Main content appears; page remains usable.
Contact intent	Open contact path	Correct approved destination.
No JavaScript	Disable scripts temporarily	Core content and contact stay usable.

Issue log template

Issue ID: ____ Device/browser: ____ Steps: ____ Expected: ____ Actual: ____

Screenshot/console: ____ Owner: ____ Fix tested on: ____

Release decision

Fix broken core journeys before polishing microdetails. A site is ready when its content is approved, primary task works, essential accessibility is checked, and the live version matches the reviewed version.

05 / BEYOND STATIC

When the site needs a back end

A static landing page can display content and link to an email address. A request for user accounts, private dashboards, online payments, or stored inquiries changes the architecture. The browser can request data, but trusted server-side code or a managed service must decide what is allowed and protect stored information.

Part	Responsibility	Example
Browser	Shows interface and sends requests.	Submit a contact inquiry.
Server / service	Validates, authorizes, processes.	Reject missing or invalid fields.
Database	Stores approved records.	Save an inquiry with restricted access.

A safe learning sequence

Learn JavaScript functions and async requests, then HTTP basics (methods, status codes, JSON), then a simple server tutorial, database fundamentals, authentication, and deployment. Build a local practice API before putting real personal data into it.

API thinking in one example

The browser sends a request to `POST /inquiries`. The server checks the fields, applies abuse controls, stores or routes the record, and returns a success or error response. The page shows “Sent” only after receiving a true success response. This is an architecture illustration, not a production-ready implementation.

When to use a managed service

If the feature involves payments, authentication, or sensitive records, compare reputable managed products and their current documentation. Review access control, data location, privacy obligations, pricing and who owns the account before building around one.

05 / FREELANCE WORKFLOW

Turn a site into a defined service

Discovery script

“Who is this page for? What action should they take? Which facts are approved? Who supplies images and copy? Who owns the domain and hosting? Who handles inquiries? What does success look like after launch?”

Scope note template

Deliverable: [pages/sections]. Includes: [responsive layout, content insertion, tests, deployment]. Client supplies: [logo, approved copy, images, contact details, legal text]. Revisions: [number and review window]. Excludes: [payments, accounts, custom backend, ongoing maintenance unless separately agreed]. Ownership and access: [written terms].

Handoff email template

“The site is live at [URL]. I tested [devices/browsers] and the contact path on [date]. Source files are at [repository/location]. Your domain/hosting owner is [name]. For updates, edit [file/CMS] and follow [steps]. Open issues: [none or list].”

Reality check

Do not quote a price or timeline before defining deliverables and dependencies. Example templates are planning aids; actual contracts, tax invoices and data practices should follow current local requirements and professional advice.

06 / 30-DAY PLAN

Days 01-15: understand and build

Days	Work	Evidence
01-03	Define user, action and three-section scope. Gather approved content.	One-page brief and content outline.
04-06	Learn file structure and semantic HTML; write the page.	Readable page without CSS.
07-09	Add typography, spacing and a narrow-screen layout.	Screen captures at two widths.
10-12	Build cards and a wider-screen layout; inspect box model.	No overflow at tested widths.
13-15	Add one JavaScript interaction and test keyboard use.	Feature works; console is clean.

Daily work block

Spend the first part reading one relevant reference, the middle building one small feature, and the end recording what changed, what failed, and what you will try next. Save a commit after a coherent change.

Checkpoint at day 15

Show the page to one person unfamiliar with the project. Ask them what the site offers and how to make contact. Record their exact confusion, then revise the copy or layout.

06 / 30-DAY PLAN

Days 16–30: verify and ship

Days	Work	Evidence
16–18	Review headings, alt text, focus, link names and contrast.	Accessibility issue list.
19–21	Check images, scripts and mobile load; fix the largest issue.	Before/after audit notes.
22–24	Review title, description, content truth and contact path.	Content approval checklist.
25–27	Commit clean code and deploy. Test the published URL.	Live link and QA log.
28–30	Write handoff, collect feedback, plan one next iteration.	Portfolio entry with scope and lessons.

Portfolio entry structure

State the problem, audience, scope, your role, the decisions you made, how you tested, and what you would improve. Use screenshots and a live link. Do not invent performance or client outcomes.

Next step

Choose a second project with a different constraint: an event page, a small portfolio, or a content-rich guide. Build from your own brief rather than copying the same template. Add a framework only after core HTML, CSS and JavaScript feel understandable.

06 / REFERENCE DESK

Quick checks and common mistakes

Mistake	Better move
Starting with a large framework	Complete one small static site first.
AI writes the whole site at once	Ask for small changes and inspect each diff.
Designing desktop only	Check narrow width from the first CSS pass.
Invented business proof	Use approved facts; label examples as illustrative.
A form that goes nowhere	Use a real handler or a direct contact link.
Deploying without QA	Test the live URL, including contact and keyboard.
Treating audits as guarantees	Use findings as diagnostic clues and retest manually.

One-minute launch brief

Audience: ____ Offer: ____ Main action: ____ Three sections: ____ Approved proof: ____ Contact path: ____ Hosting owner: ____ Reviewer: ____ Launch date: ____

SOURCES & FURTHER READING

Official references

Reviewed for this edition in September 2026. Recheck live documentation for changing interfaces and rules.

MDN Learn Web Development

https://developer.mozilla.org/en-US/docs/Learn_web_development

Core HTML, CSS, JavaScript and beginner sequence.

MDN Forms and validation

https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation

Client-side validation and form behavior.

MDN Web Security

<https://developer.mozilla.org/en-US/docs/Web/Security>

HTTPS, input handling and security concepts.

W3C WAI forms and images

<https://www.w3.org/WAI/tutorials/forms/labels/>

Labels and accessible form controls.

W3C WAI images

<https://www.w3.org/WAI/tutorials/images/>

Purposeful text alternatives.

web.dev Learn Performance

<https://web.dev/learn/performance>

Measurement and performance improvements.

Google Search Central SEO Starter Guide

<https://developers.google.com/search/docs/fundamentals/seo-starter-guide>

Search-friendly content and structure.

GitHub Pages Quickstart

<https://docs.github.com/en/pages/quickstart>

Current static-site publishing steps.

LEARNORA

Build something useful. Understand how it works.

Your next move: open a blank folder, write the three-section brief, and create index.html. Small working changes become a site you can maintain.

LEARN TODAY. EARN TOMORROW.

Educational guide • No guaranteed professional or financial outcome